TNV HUB PORTAL

Backend and Database Specification

Phase 1 Demo | Supabase (Postgres, Auth, Storage, Edge Functions)

Internal engineering document | Version 1.0 | 20 October 2026 | Demo: Thursday 22 October 2026

Contents

1. Purpose and scope
2. System architecture
3. Ownership: backend and database
4. Database design
5. Row-level security
6. Backend: Edge Functions (with flows)
7. Database: RPC contracts
8. Authentication flow
9. End-to-end system flow and status lifecycle
10. Business rules
11. Security requirements
12. Interfaces to agree between the two engineers
13. Seed data and test accounts
14. Checklists
15. Verification script
16. Schedule and open items

Internal. Contains design for systems that hold Aadhaar data. Do not share outside the development team.

1. Purpose and scope

This document defines the Phase 1 backend and database for the TNV Hub Portal: what is built, who builds it, how the parts connect, and the rules the system must enforce. It is written for the two engineers who own the server side. Phase 1 must be demonstrable on **Thursday 22 October 2026**.

In scope

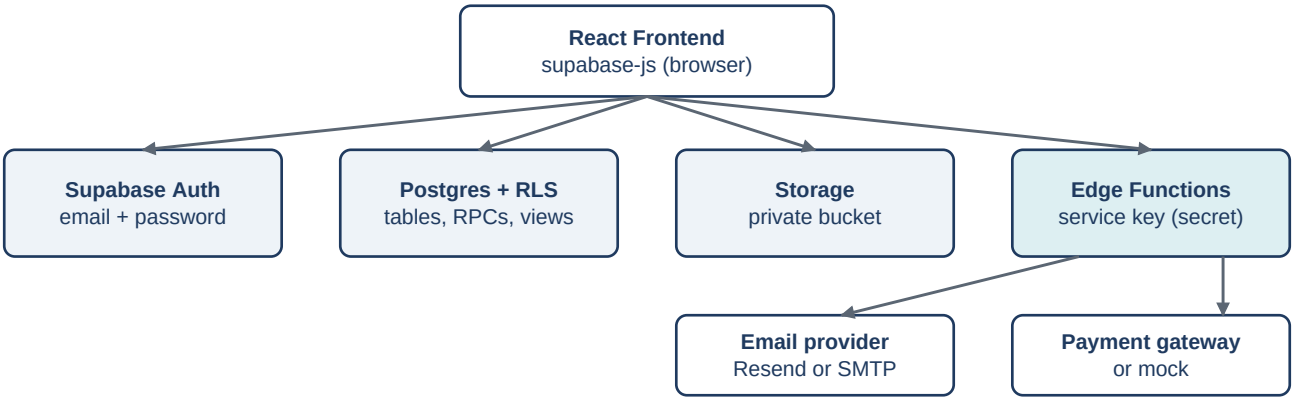
- Supabase project, secrets, storage bucket and email delivery
- Postgres schema, enums, constraints, helper functions, views, row-level security and seed data
- Edge Functions: register, record_payment, approve_applicant, create_donation
- Database RPCs: username_available, login_email_for, log_call, save_scores, reveal_aadhaar, my_links
- Security checks and the end-to-end verification script

Out of scope

- Frontend screens: landing page (T1), applicant dashboard states (T4), ID card and appointment letter (T9). The backend only has to supply the data they need.
- Phase 2 items: password-less login, background follow-up reminders, department admin management screens.

2. System architecture

Supabase provides Postgres, Auth, Storage and Edge Functions. The React app talks to Supabase directly. Row-level security (RLS) in Postgres is the access-control layer: the frontend only hides what the database already refuses. Anything that crosses a trust boundary (creating accounts, taking payments, changing approval status, sending email) runs in an Edge Function.



Edge Functions also write to Postgres, Auth and Storage with the service key. The browser never holds the service key. RLS is the access-control layer.

Concern	Approach
Authentication	Supabase Auth with email and password. The user chooses a username; login resolves it to the email through a database function, then signs in with email and password.
Data	Postgres tables in section 4, all with RLS enabled. Plain reads and writes go through supabase-js under RLS.
Privileged actions	Edge Functions using the service key: send email, record payments, approve applicants. The browser never holds the service key.
Email	Transactional email from an Edge Function (Resend or SMTP): welcome and approval. Password reset uses Supabase built-in email.
Files	Private storage bucket for receipts and photos. Download through signed URLs only.
Roles	profiles.role plus an is_admin() SQL helper used by every admin policy.

Key design decisions

- **One registration path, one table.** Every applicant is a row in profiles. Status (pending or approved) and role are columns, not separate user types.
- **No passwords by email.** The original brief says credentials are sent by email, but applicants choose their own password. The welcome email carries the username and a login link only.
- **Unpaid applicants stay.** Payment is never a condition for creating a profile or entering the admin queue.
- **Admin-only data is isolated.** Call logs, scores and notes live in separate tables with no applicant policy at all.
- **Aadhaar is protected.** Stored encrypted, displayed masked (last 4 digits). Only a Master Admin reveal action shows the full number, and every reveal is logged.

3. Ownership: backend and database

The split below follows the specification: items the spec calls Edge Functions belong to the backend engineer, items it calls RPCs, tables or policies belong to the database engineer. Confirm the split with the team lead on day one.

Area	Backend engineer	Database engineer
Project setup	Secrets, environment keys outside the repo, service key only in Edge Function secrets	Supabase project settings, backups or point-in-time recovery
Schema	Review and consume	Enums, all tables, constraints, is_admin(), profiles_public view
Access control	Verify caller identity and role inside every Edge Function	RLS policies, protection of status, role, tnv_id and Aadhaar columns
Functions	register, record_payment, approve_applicant, create_donation	username_available, login_email_for, log_call, save_scores, reveal_aadhaar, my_links
Identifiers	Call the generators from functions	Sequences or functions for tnv_id and receipt_no
Email and files	Email provider, welcome and approval emails, receipt PDF, storage bucket and signed URLs	Seed WhatsApp link rows only
Aadhaar	Validation and encryption at registration (see section 12)	Masked display, reveal function, audit row
Seed data	Provide test applicant needs	Districts, wings, sectors, questions, links, Master Admin, 3 test applicants
Testing	Function tests and the end-to-end script	RLS tests and constraint tests

4. Database design

Run the SQL in the order given in the specification. Types and names are final for Phase 1: add columns later, never rename.

4.1 Types

Type	Values
app_status	pending, approved
app_role	volunteer, district_coordinator, wing_lead, dept_admin, master_admin
call_outcome	no_answer, scheduled, attended
pay_status	paid, failed

4.2 Tables and visibility

Table	Holds	Visible to
profiles	One row per person: identity, contact, district, status, role, TNV ID	Owner (own row), admins (all)
districts, wings, sectors	38 districts, wings (Administration, Online), 13 sectors	Everyone (read)
whatsapp_links	Group links by district, wing, sector, plus announcements and social	Approved users, admins
payments	Membership payments and receipts	Owner, admins
donations	Open donations from Rs 1, may be anonymous	Admins (insert through Edge Function)
call_logs	Each call attempt: outcome, callback time, note, admin	Admins only
interview_questions	About 10 ordered, editable questions	Admins only
interview_scores	Score per question per applicant (0, 0.5, 1)	Admins only
verifications	Final decision: total score, role, approved by, time	Admins (owner reads outcome via profile)
notifications	In-app alerts to admins (new registration)	Admins
audit_log	Sensitive actions: Aadhaar reveal, approvals, role changes	Master Admin

Relationships: profiles 1-N call_logs, 1-N interview_scores, 1-N payments, 1-1 verifications. profiles N-1 districts. interview_scores N-1 interview_questions. whatsapp_links N-1 districts, wings and sectors.

4.3 profiles (core table)

Column	Type	Constraint and notes
id	uuid, PK	References auth.users(id), cascade delete
username	text, unique	Check: 4 to 20 letters, digits or underscore
full_name	text, not null	
email	text, not null	Must match the Auth user email
phone	text, not null	Check: 10 digits starting 6 to 9
address	text	Optional
district_id	int, not null	References districts(id)
aadhaar_enc	text, not null	Encrypted. Never selected by client code
aadhaar_last4	char(4), not null	For masked display
status	app_status, not null	Default pending. Changed only by the approval function
role	app_role	Null until verified
wing_id	int	References wings(id)
tnv_id	text, unique	Generated on approval
photo_path	text	Storage path
membership_paid	boolean, not null	Default false. Set by record_payment
created_at	timestamptz	Default now()

4.4 Supporting tables (key columns and constraints)

Table	Key columns	Constraints
payments	id, profile_id, amount_inr, status, gateway_ref, receipt_no, paid_at	amount_inr must equal 365. receipt_no unique.
donations	id, donor_name, message, amount_inr, gateway_ref, created_at	amount_inr at least 1. donor_name nullable for anonymous donors.
call_logs	id, profile_id, admin_id, outcome, callback_at, note, created_at	callback_at required when outcome is scheduled. Cascade delete with profile.
interview_questions	id, position, text, active	Only active questions count for scoring.
interview_scores	profile_id, question_id, score, note, scored_by	Primary key (profile_id, question_id). score in 0, 0.5, 1.
verifications	profile_id, total_score, recommended_role, approved_by, approved_at	One row per profile.
whatsapp_links	id, kind, district_id, wing_id, sector_id, label, url	kind in district, wing, sector, announcements, social.
notifications	id, kind, profile_id, read, created_at	
audit_log	id, actor, action, target, detail (jsonb), at	Append only in practice.

4.5 Helper function

is_admin() returns true when the calling user has role master_admin or dept_admin. It is declared security definer so policies can call it without recursion on profiles.

Recommendation. Declare is_admin() with a fixed search_path (for example set search_path = public) because it is security definer. Also note it returns true for dept_admin as well as master_admin. Anything the spec restricts to the Master Admin (audit_log, reveal_aadhaar) needs an explicit master_admin check, not is_admin().

5. Row-level security

Enable RLS on every table. Principle: deny by default, then allow the minimum. A table with RLS enabled and no policy returns nothing.

Table	Applicant (own row)	Admin
profiles	Select own row. Update only name, phone, address, photo. Cannot change status, role, tnv_id or Aadhaar columns.	Select and update all
payments	Select own	Select all
call_logs, interview_questions, interview_scores, verifications, audit_log, notifications	No access. No policy exists for applicants.	Select, insert, update (audit_log: Master Admin only)
whatsapp_links	Select only if own profile status is approved	All
districts, wings, sectors	Select	Select
donations	No policy. Inserts happen only inside the create_donation Edge Function.	Select

Rules that must hold

- Aadhaar columns are never returned to the browser for applicants or normal queries. Expose a **profiles_public** view without them. The only way to see the full number is the reveal_aadhaar function.

- Applicants cannot self-approve. status and role are writable only by the approval path.
- dept_admin rows are created later by the Master Admin. For Thursday only the master_admin exists, and the policies must already allow adding dept admins with no schema change.

Critical. RLS decides which rows a user may touch, not which columns. A policy such as "update own row" would still let an applicant set their own status or role. The database engineer must add a column-level control: either column-level UPDATE grants (grant update only on name, phone, address, photo_path) or a BEFORE UPDATE trigger that rejects changes to status, role, tnv_id, wing_id, membership_paid and the Aadhaar columns unless the caller is the service role. This is verified in the security checklist.

Spec inconsistency. The entity overview in the original spec says donations can be inserted by anyone, while the RLS table says Edge Function only. Use the stricter reading: no public insert policy on donations. The Edge Function inserts with the service key, which also lets it validate the amount and the payment first.

6. Backend: Edge Functions

Four Edge Functions run with the service key. Each function must verify the caller itself, because the service key bypasses RLS. Never trust a user id, amount or role sent in the request body; derive them from the verified token or from the server.

Function	Caller	Task	Purpose
register	Anonymous visitor	T2	Validate, create auth user and profile, encrypt Aadhaar, notify admins, send welcome email, return a session
record_payment	Signed-in applicant	T3	Verify payment, store it, set membership_paid, issue receipt
approve_applicant	Signed-in admin	T7, T8	Check scoring is complete, approve, assign role and TNV ID, email, audit
create_donation	Anyone, signed out	T12	Validate amount, take payment, store donation

6.1 register

Type	Edge Function, public (no login required)
Task	T2
Input	username, full_name, email, phone, address (optional), district_id, aadhaar, password
Output	Session on success. A clear field-level error message on failure.
Writes	auth.users, profiles, notifications. Sends one email.

#	Step	What happens
1	**Validate input**	Username matches 4 to 20 letters, digits or underscore. Phone matches 10 digits starting 6 to 9. Aadhaar is exactly 12 digits. District exists. Password meets the minimum length rule. Reject with a clear message on the first failure; create nothing.
2	**Check duplicates**	Username already in profiles, or email already in Auth: reject with a clear message that names the field.
3	**Create auth user**	Create the user with the email and the password the applicant chose. The password is hashed by Supabase Auth and never stored or logged by your code.
4	**Encrypt Aadhaar**	Encrypt the number, keep the encrypted value in aadhaar_enc and the last four digits in aadhaar_last4. The plain number is never stored, logged or returned.
5	**Insert profile**	Insert the profiles row with the Auth user id, status pending, role null, membership_paid false.
6	**Notify admins**	Insert a notifications row of kind new_registration pointing at the profile.

#	Step	What happens
7	**Send welcome email**	Email contains the username and a login link. It never contains the password.
8	**Return session**	Return the session so the applicant is signed in immediately.

Failure handling. If any step after the Auth user is created fails, delete the Auth user (the profile cascades) so no half-created account remains. A failed welcome email should be logged and retried, not treated as a registration failure.

6.2 record_payment

Type	Edge Function, requires a signed-in user
Task	T3
Input	Gateway reference or mock marker. The amount is not accepted from the client.
Output	receipt_no and a signed URL for the receipt PDF on success. Failure status otherwise.
Writes	payments, profiles.membership_paid, storage (receipt PDF)

#	Step	What happens
1	**Identify caller**	Take the profile id from the verified token. Never from the request body.
2	**Verify the payment**	Check the gateway result on the server with the gateway, or accept the mock marker in demo mode.
3	**Fix the amount**	The amount is Rs 365, set by the server.
4	**If payment failed**	Optionally record a payments row with status failed. Change nothing else. Registration and queue entry are unaffected.
5	**If payment succeeded**	Generate receipt_no in the form TNV-R-<year>-<running number> from a database sequence. Insert payments with status paid. Set profiles.membership_paid to true.
6	**Generate receipt**	Create the receipt PDF and upload it to the private receipts bucket under a path derived from profile id and receipt_no.
7	**Return**	Return receipt_no and a short-lived signed URL.

Recommendation. Treat gateway_ref as an idempotency key: if the same gateway_ref arrives twice, return the existing receipt instead of creating a second one. If membership_paid is already true, return the existing receipt. These are recommendations, not stated in the spec.

6.3 approve_applicant

Type	Edge Function, admin only
Task	T7, T8
Input	profile_id, chosen role, wing_id (optional)
Output	Updated profile summary (status, role, tnv_id, total score)
Writes	profiles, verifications, audit_log. Sends one email.

#	Step	What happens
1	**Verify the admin**	Check the token and confirm the caller has role master_admin or dept_admin.
2	**Load the applicant**	Reject if the profile does not exist or is already approved. There is no path back to pending in Phase 1.
3	**Check scoring is complete**	Every active interview question must have a score for this applicant. If any is missing, refuse and list the missing questions.

#	Step	What happens
4	**Compute total**	total_score is the sum of the scores. Maximum equals the number of active questions.
5	**Generate TNV ID**	From a database sequence. Placeholder format: TNV-<district code>-<running number> (final format pending).
6	**Commit**	In one transaction: set status approved, role, wing_id and tnv_id on the profile, and insert the verifications row (total_score, chosen role, approved_by, approved_at). The admin's chosen role is what is saved, even if it differs from the suggestion.
7	**Write audit**	Insert an audit_log row with actor, action, target and detail (score, role, tnv_id).
8	**Send approval email**	Welcome the new member and link to the login page. Email failure is logged, not rolled back.
9	**Return**	Return the updated summary so the admin screen can confirm.

Recommendation. Make step 6 atomic by calling one SQL function from the Edge Function, so a failure cannot leave a half-approved profile. Also restrict assignable roles to volunteer, district_coordinator and wing_lead in Phase 1, since dept_admin and master_admin are created by the Master Admin separately.

6.4 create_donation

Type	Edge Function, public (signed-out donors allowed)
Task	T12
Input	amount_inr, donor_name (optional), message (optional), anonymous flag
Output	Confirmation. Nothing is returned that exposes other donations.
Writes	donations

#	Step	What happens
1	**Validate**	amount_inr is a whole number of at least 1. Limit the length of name and message.
2	**Take payment**	Charge through the gateway, or accept the mock marker in demo mode.
3	**Verify result**	Confirm success on the server before writing anything.
4	**Insert donation**	Insert with the service key. If anonymous, store donor_name as null.
5	**Return**	Return a simple confirmation to the donor.

Rule. Donations are visible only to admins. Applicants and anonymous visitors have no read policy on this table.

7. Database: RPC contracts

These functions live in Postgres and are called by the client through supabase-js. Each admin function must check the caller's role inside the function, not rely on the client.

Function	Caller	Contract	Validation and rules
username_available(u)	Anonymous	Returns true if the username is free	Check format first. Return only a boolean.
login_email_for(username)	Anonymous	Returns the email for the username, or nothing for an unknown name	Same response shape for unknown names. Never reveal which part failed.
log_call(profile_id, outcome, callback_at, note)	Admin	Inserts a call_logs row	admin_id is auth.uid(). Outcome scheduled requires callback_at. No limit on attempts.

Function	Caller	Contract	Validation and rules
save_scores(profile_id, scores)	Admin	Upserts scores for an applicant	Each score must be exactly 0, 0.5 or 1. Question must be active. scored_by is auth.uid().
reveal_aadhaar(id)	Master Admin	Returns the decrypted Aadhaar	Master Admin check (not is_admin). Writes an audit_log row in the same call.
my_links()	Approved user	Returns own district link, wing link, all sector links, announcements, social	Returns zero rows for a pending user. Derives district from the caller's profile.

Risk. login_email_for returns the email address belonging to a username to anonymous callers. This is required by the design, but it allows someone to harvest emails for known usernames. Apply rate limiting on this call (for example through an Edge Function wrapper or Supabase API limits) and never return any other field.

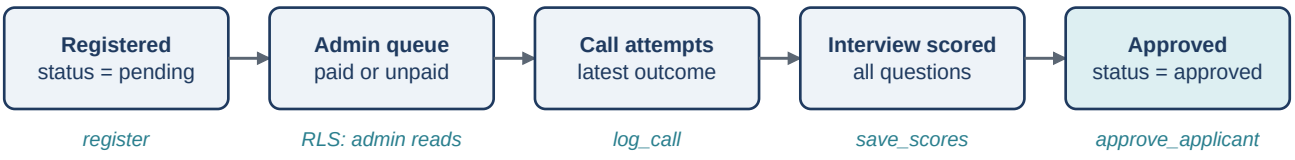
8. Authentication flow

Login uses username and password. Supabase Auth only understands email, so the client resolves the username first.

#	Step	What happens
1	**Enter credentials**	User enters username and password on the login page.
2	**Resolve email**	Client calls login_email_for(username). It returns the email, or nothing.
3	**Sign in**	Client calls Supabase Auth sign-in with the email and the password.
4	**Handle errors**	Unknown username and wrong password return the **same generic message**, so usernames cannot be discovered.
5	**Session**	A valid session is returned. The app reads the profile (status, role) through profiles_public and routes accordingly.
6	**Password reset**	Supabase built-in reset email with a link to set a new password. Used when the user clicks Forgot password.

9. End-to-end system flow and status lifecycle

9.1 Status lifecycle



One direction only in Phase 1. There is no path from approved back to pending.

9.2 Backend flow by phase

Phase	Trigger	Component	Reads and writes	Result
Username check	User types a username	username_available	Reads profiles	Available or taken
Registration	Form submit	register (Edge Function)	Writes auth.users, profiles, notifications. Sends welcome email	Pending profile, session
Payment	Pay Rs 365	record_payment (Edge Function)	Writes payments, membership_paid, receipt in storage	Receipt, or no change on failure
Login	Username and password	login_email_for, Supabase Auth	Reads profiles, Auth	Session
Admin review	Admin opens queue	RLS on profiles, notifications	Reads all profiles, masked Aadhaar	Queue with paid and unpaid
Aadhaar reveal	Master Admin clicks Reveal	reveal_aadhaar	Reads aadhaar_enc, writes audit_log	Full number shown, reveal logged
Call handling	Admin logs an attempt	log_call	Writes call_logs	Latest outcome is current call status
Scoring	Admin scores questions	save_scores	Writes interview_scores	Running total
Approval	Admin marks Verified	approve_applicant (Edge Function)	Writes profiles, verifications, audit_log. Sends approval email	Approved, role, TNV ID
Member access	Approved user opens links	my_links, RLS on whatsapp_links	Reads whatsapp_links	Own district, wing, sectors, announcements, social
Donation	Donor submits amount	create_donation (Edge Function)	Writes donations	Recorded, visible to admins only

10. Business rules

Area	Rule
Status	A new profile is pending. Only approve_applicant moves it to approved. No path back in Phase 1.
Call log	Unlimited attempts per applicant. The latest outcome is the applicant's current call status in the queue. Scheduled requires a callback time.
Scoring	Each score is 0, 0.5 or 1. Total is the sum. Maximum equals the number of active questions. The role recommendation is a suggestion; the admin's choice is saved.
TNV ID	Generated at approval. Placeholder format TNV-<district code>-<running number>. Final format pending confirmation.
Payment	Fixed at Rs 365. Failure never blocks registration. Receipt number format TNV-R-<year>-<running number>.
Donations	Any whole amount from Rs 1. May be anonymous.
Follow-up	A query lists scheduled callbacks that are due and no-answer applicants older than 24 hours. An admin queue filter is enough for Thursday.
Unpaid applicants	Always created and always visible in the admin queue.

11. Security requirements

#	Requirement	Owner
1	RLS enabled on every table. A table without policies returns nothing.	Database
2	An applicant cannot read other profiles, call logs, scores or verifications.	Database
3	An applicant cannot update own status, role, tnv_id or Aadhaar columns, even by calling the API directly.	Database
4	A pending applicant gets zero rows from whatsapp_links.	Database
5	Aadhaar never appears in any applicant or list query. Reveal writes an audit row.	Both
6	The service key exists only in Edge Function secrets. Never in the frontend or the repository.	Backend
7	Every Edge Function verifies the caller and derives user id, role and amount on the server.	Backend
8	Passwords are never emailed, logged or stored by application code.	Backend
9	Backup or point-in-time recovery is on before any real data is entered.	Database
10	Login failures return one generic message for unknown username and wrong password.	Both

12. Interfaces to agree between the two engineers

Settle these before writing code. Most delays come from assumptions here.

Item	Question to settle	Suggested default
Names	Final table, column and function names	Use the spec names exactly. Add columns later, never rename.
Aadhaar encryption	Where is the number encrypted and decrypted, and where is the key held?	Encrypt and decrypt in the same layer with the same key. Keep the key in secrets or Vault, never in code or tables.
Column protection	Who protects status, role, tnv_id and Aadhaar columns from applicant updates?	Database engineer, using column grants or a trigger. Backend functions use the service role.
Identifiers	How are tnv_id and receipt_no generated?	Database sequences, called by the Edge Functions. Format changes later should need no schema change.
Atomic approval	Can approval be one SQL function called by the Edge Function?	Yes. Database engineer writes it, backend calls it.
Storage paths	Receipt and photo paths	receipts/<profile_id>/<receipt_no>.pdf and photos/<profile_id>/...
Payment mode	Real gateway or mock for the demo	Mock for Thursday if the gateway account is not ready. Keep the same function interface.
Test accounts	Master Admin and three test applicants	Database engineer seeds them by Tuesday evening.
Admin RPC checks	Role checks inside each admin RPC	Each RPC checks is_admin(), or master_admin where required.

13. Seed data and test accounts

- 38 Tamil Nadu districts.
- Wings: Administration and Online.
- 13 sectors. **Names to be confirmed with Sir.**
- WhatsApp links: real links if available, otherwise clearly fake placeholders. At least three districts must have different links.

- About 10 interview questions copied from the TNV Leadership Selection Portal.
- One Master Admin account for Sir.
- Three test applicants: one paid, one unpaid, one already approved.

14. Checklists

Build items first, then the security items. Do not skip the security items for the demo: they are quick and protect real Aadhaar data.

14.1 Backend engineer

Done	Check	Ref
[ ]	Secrets configured (service key, email provider key, encryption key). Nothing in the repo.	Setup
[ ]	Private storage bucket created; signed URL download works	Setup
[ ]	register creates auth user, profile and admin notification in one call	T2
[ ]	Duplicate username or email rejected with a clear message	T2
[ ]	Bad Aadhaar (11 digits) and bad phone rejected before anything is created	T2
[ ]	Welcome email arrives with username and login link, no password	T2
[ ]	Password reset email arrives and the new password works	T2
[ ]	record_payment stores the payment, sets membership_paid, returns receipt; failed payment changes nothing else	T3
[ ]	approve_applicant refuses when any active question is unscored	T7
[ ]	approve_applicant sets status, role and TNV ID, writes verification, audit row, and sends approval email	T8
[ ]	create_donation accepts Rs 1, accepts anonymous, rejects 0 and negatives	T12

14.2 Database engineer

Done	Check	Ref
[ ]	All enums and tables created without errors, in order	Schema
[ ]	Districts, wings, sectors, questions and WhatsApp links seeded	Seed
[ ]	RLS enabled on every table	RLS
[ ]	profiles_public view excludes the Aadhaar columns	RLS
[ ]	Applicant cannot update status, role, tnv_id (tested directly through the API)	RLS
[ ]	Pending applicant gets zero rows from whatsapp_links	RLS
[ ]	log_call saves all three outcomes; scheduled without a time is rejected	T6
[ ]	save_scores accepts 0, 0.5, 1 and rejects every other value	T7
[ ]	reveal_aadhaar works for Master Admin only and writes an audit row	T5
[ ]	my_links returns the right district link for users in three different districts	T10
[ ]	Backup or point-in-time recovery turned on	Ops

15. Verification script

Run the full script twice before the demo: once as Sir will see it and once in a private window on a phone. Every step must pass without touching the database by hand. Use a new username each run.

#	Action	Backend or database behaviour verified
1	Submit form with an 11-digit Aadhaar	register rejects; nothing created
2	Submit valid details	Auth user, profile, notification created; welcome email without password
3	Skip payment	Profile stays pending; membership_paid false
4	Log out and log in with username and password	login_email_for and Auth sign-in; pending state
5	Open admin URL as the applicant	Refused by RLS and role checks
6	Log in as Master Admin	Notification present; applicant listed as unpaid; Aadhaar masked
7	Log No Answer, then Scheduled with time, then Attended	log_call accepts all three; scheduled without time rejected
8	Score all questions with 0, 0.5 and 1	save_scores accepts; total correct
9	Mark Verified	approve_applicant: status, role, TNV ID, verification, audit, approval email
10	Applicant dashboard refresh	Status approved visible through profiles_public
11	Open WhatsApp section	my_links returns own district link, wing, 13 sectors, announcements, social
12	Repeat with a paid applicant	record_payment: receipt generated and downloadable
13	Donate Rs 1 while signed out	create_donation records it; applicants cannot read it
14	Master Admin reveals Aadhaar	reveal_aadhaar returns number; audit_log row written

16. Schedule and open items

16.1 Schedule

When	Backend engineer	Database engineer
Tuesday 20 Oct	Setup and secrets. Agree interfaces (section 12). Build register, then record_payment and the emails.	Schema, RLS, column protection, profiles_public, sequences, seed data, test accounts.
Wednesday morning	approve_applicant, create_donation, storage and receipt PDF.	All RPCs, backups, RLS tests.
Wednesday afternoon	Both end-to-end runs. Fix defects together.	Both end-to-end runs. Fix defects together.
Thursday 22 Oct	Smoke test only. No new changes.	Smoke test only. No new changes.

16.2 Open items

Item	Needed from	Impact
13 sector names	Sir	Seed data, my_links
Final TNV ID format	Sir	ID generation. Placeholder in use.
Real WhatsApp links or placeholders	Sir	Seed data, link demo
Real payment gateway or mock	Team lead	record_payment, create_donation
Whether the profile page offers payment later	Sir and frontend	record_payment called after registration. Backend supports it either way.
Aadhaar encryption layer and key location	Both engineers	register and reveal_aadhaar